

在 Go 中檢測應用程式，提升效能 (第 1 部分：追蹤記錄)

來源：<https://codelabs.developers.google.com/codelabs/otel-cloudtrace-go?hl=zh-tw>

步驟數：8

OpenTelemetry 是業界觀測追蹤記錄和指標的業界標準。另外，持續剖析是辨識最後一英里的資訊，以利調整效能。在本程式碼研究室中，您將瞭解如何使用 OpenTelemetry 檢測應用程式，以用於追蹤記錄和分析器代理程式，以及如何在 Cloud Trace 和 Cloud Profiler 的視覺化圖表中找出瓶頸。

目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 建構、推送及部署微服務](#)
4. [4. HTTP 的檢測](#)
5. [5. gRPC 的檢測](#)
6. [6. 使用 Cloud Trace 視覺化追蹤記錄](#)
7. [7. 新增子範圍，以便進行更完善的分析](#)
8. [8. 恭喜](#)

1. 簡介



上次更新時間：2022 年 7 月 15 日

應用程式的觀測能力

觀測能力和 OpenTelemetry

「可觀測性」一詞用於描述系統的屬性。如果系統具備可觀測性，團隊就能主動偵錯。在這種情況下，可觀測性的三大支柱 (記錄、指標和追蹤記錄) 是系統取得可觀測性的基本工具。

OpenTelemetry 是一組規格、程式庫和代理程式，可加速遙測資料 (記錄、指標和追蹤記錄) 的檢測和匯出，以滿足可觀測性需求。OpenTelemetry 是 CNCF 旗下由社群主導的開放標準專案。開發人員可以運用專案及其生態系統提供的程式庫，以廠商中立的方式針對多種架構檢測應用程式。

此外，除了可觀測性的三大支柱外，持續分析也是可觀測性的**另一個重要元件**，且**使用者族群正在擴大**。Cloud Profiler 是其中一個發起者，提供簡單的介面，可下鑽應用程式呼叫堆疊中的效能指標。

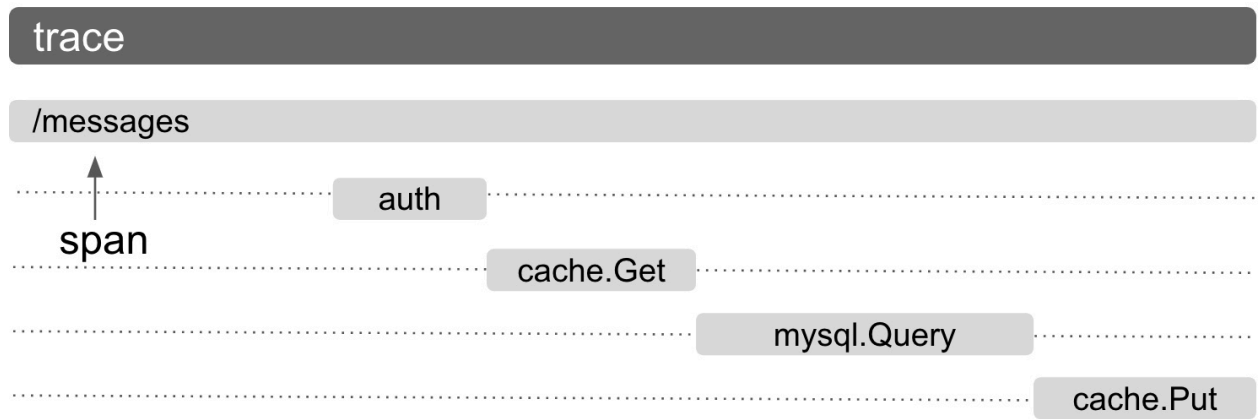
本程式碼實驗室是系列的第一部分，涵蓋如何使用 OpenTelemetry 和 Cloud Trace，在微服務中檢測分散式追蹤記錄。第 2 部分將介紹如何使用 Cloud Profiler 持續剖析。

分散式追蹤

在記錄、指標和追蹤記錄中，追蹤記錄是遙測資料，可顯示系統中特定程序部分的延遲時間。尤其在微服務時代，分散式追蹤是找出整體分散式系統延遲瓶頸的強大推手。

分析分散式追蹤記錄時，追蹤記錄資料的視覺化呈現方式是掌握整體系統延遲時間的關鍵。在分散式追蹤中，我們會處理一組呼叫，以處理系統進入點的單一要求，形式為包含多個時距的追蹤記錄。

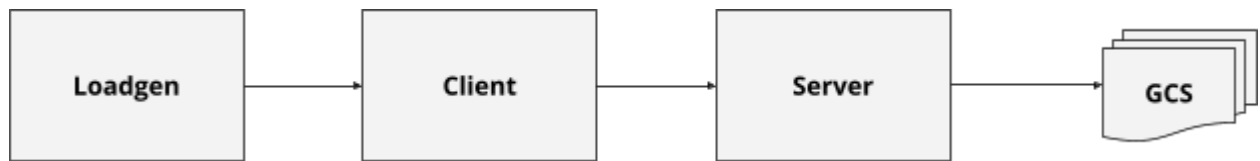
時距代表分散式系統中完成的個別工作單元，會記錄開始和停止時間。跨度之間通常具有階層關係，如下圖所示，所有較小的跨度都是大型 /messages 跨度的子項跨度，並組合成一個追蹤記錄，顯示工作在系統中的路徑。



Google Cloud Trace 是分散式追蹤後端的選項之一，且與 Google Cloud 中的其他產品整合良好。

建構項目

在本程式碼研究室中，您將在 Google Kubernetes Engine 叢集上執行的「Shakespeare 應用程式」（又稱 Shakesapp）服務中，安插追蹤資訊。Shakesapp 的架構如下所述：



- Loadgen 會以 HTTP 格式將查詢字串傳送至用戶端
- 用戶端會透過 gRPC 將查詢從 loadgen 傳遞至伺服器
- 伺服器接受用戶端查詢，從 Google Cloud Storage 擷取所有莎士比亞作品的文字格式，搜尋包含查詢的行，並將符合的行號傳回給用戶端

您會在要求中記錄追蹤資訊。之後，您會在伺服器中嵌入剖析器代理程式，並找出效能瓶頸。

課程內容

- 如何在 Go 專案中開始使用 OpenTelemetry Trace 程式庫
- 如何使用程式庫建立範圍
- 如何在應用程式元件之間的線路傳播範圍內容
- 如何將追蹤資料傳送至 Cloud Trace
- 如何在 Cloud Trace 上分析追蹤記錄

本程式碼研究室說明如何檢測微服務。為方便瞭解，這個範例只包含 3 個元件（負載產生器、用戶端和伺服器），但您可以將本程式碼研究室中說明的相同程序，套用至更複雜的大型系統。

軟硬體需求

- Go 的基本知識
- Kubernetes 的基本知識

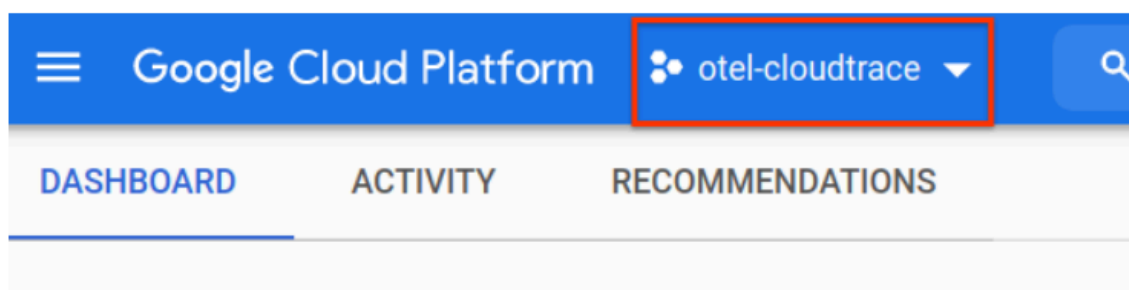
步驟 2 · 約 0 分鐘

2. 設定和需求

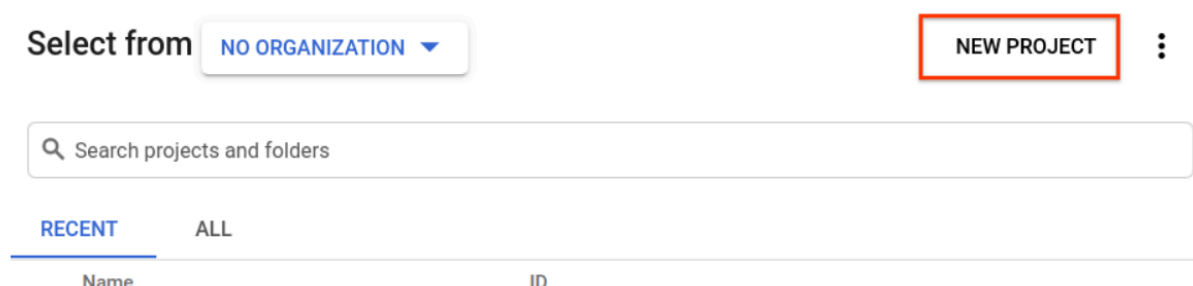
自修實驗室環境設定

如果您沒有 Google 帳戶 (Gmail 或 Google 應用程式)，請先[建立帳戶](#)。登入 Google Cloud Platform 主控台 (console.cloud.google.com)，然後建立新專案。

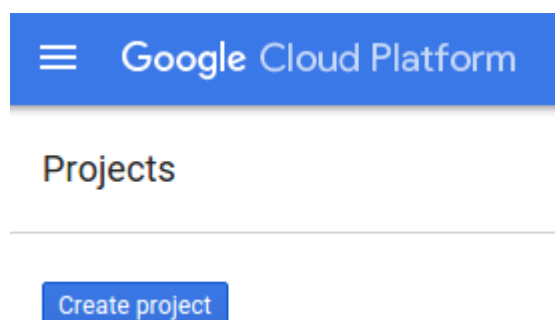
如果您已有專案，請按一下主控台左上方的專案選取下拉式選單：



然後在隨即顯示的對話方塊中，按一下「NEW PROJECT」(新專案) 按鈕，建立新專案：



如果您還沒有專案，應該會看到如下對話方塊，請建立第一個專案：



後續的專案建立對話方塊會顯示，讓您輸入新專案的詳細資料：

New Project

Project name *

otel-cloudtrace



Project ID: otel-cloudtrace-306615. It cannot be changed later. [EDIT](#)

Billing account *



Any charges for this project will be billed to the account you select here.

Organization *

請記住專案 ID，所有 Google Cloud 專案的專案 ID 都是不重複的名稱 (上述名稱已遭占用，因此不適用於您，抱歉！)。本程式碼研究室稍後會將其稱為 PROJECT_ID。

接下來，如果尚未啟用，您需要在開發人員控制台中[啟用帳單](#)，才能使用 Google Cloud 資源並[啟用 Cloud Trace API](#)。

Billing

This project has no billing account

This project is not linked to a billing account

[Link a billing account](#)

or

[Manage billing accounts](#)

完成本程式碼研究室的費用不應超過數美元，但如果您決定使用更多資源，或是將資源繼續執行 (請參閱本文件結尾的「清除」一節)，則可能會增加費用。Google Cloud Trace、Google Kubernetes Engine 和 Google Artifact Registry 的價格請參閱官方說明文件。

- [Google Cloud 作業套件的計價方式 | 作業套件](#)
- [定價 | Kubernetes Engine 說明文件](#)
- [Artifact Registry 定價 | Artifact Registry 說明文件](#)

Google Cloud Platform 新使用者享有價值 **\$300 美元的免費試用期**，因此本程式碼研究室應完全免費。

Google Cloud Shell 設定

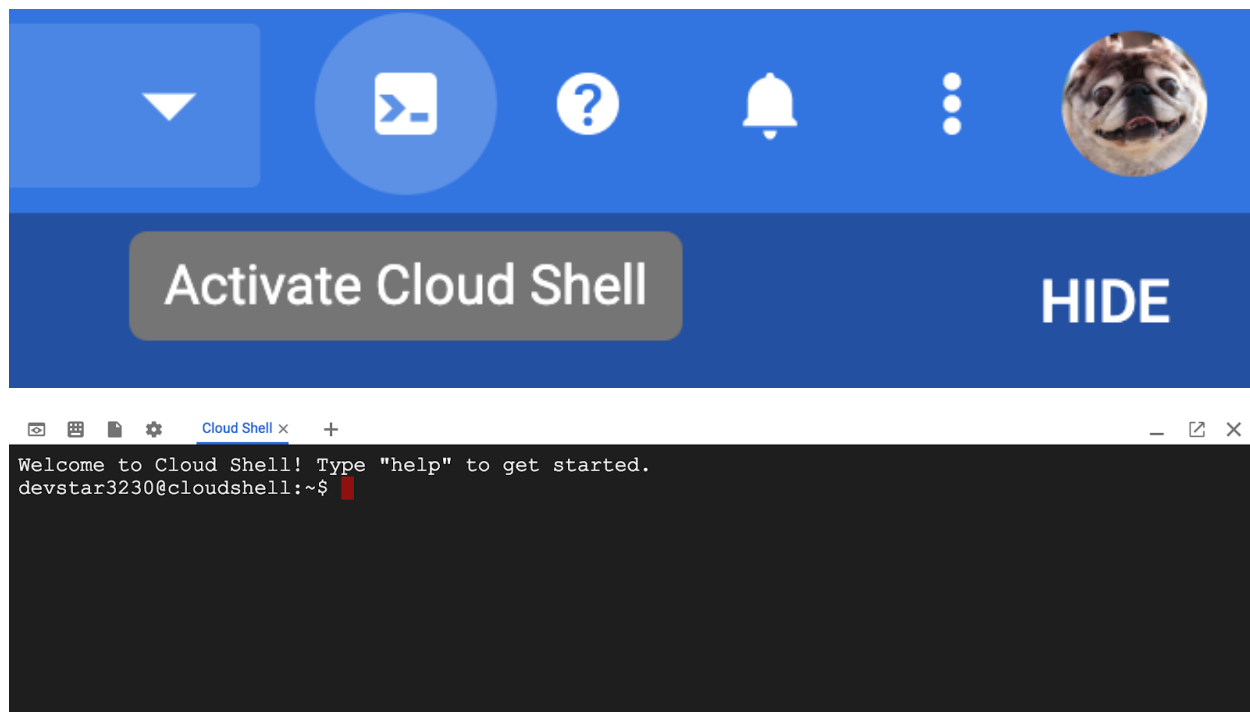
雖然可以透過筆電遠端操作 Google Cloud 和 Google Cloud Trace，但在本程式碼研究室中，我們將使用 [Google Cloud Shell](#)，這是 Cloud 中執行的指令列環境。

這部以 Debian 為基礎的虛擬機器，搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。也就是說，您只需要瀏覽器 (Chromebook 也可以) 就能完成本程式碼研究室。

如要從 Cloud 控制台啟用 Cloud Shell，只要按一下「啟用 Cloud Shell」圖示



即可 (佈建並連線至環境的作業需要一些時間才能完成)。



連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID `PROJECT_ID`。

```
gcloud auth list
```

指令輸出

```
Credentialed accounts:
- <myaccount>@<mydomain>.com (active)
```

注意：Cloud Shell 已預先安裝 `gcloud` 指令列工具，並支援 Tab 鍵完成功能，相信您一定會喜歡。詳情請參閱 [gcloud 指令列工具總覽](#)。

```
gcloud config list project
```

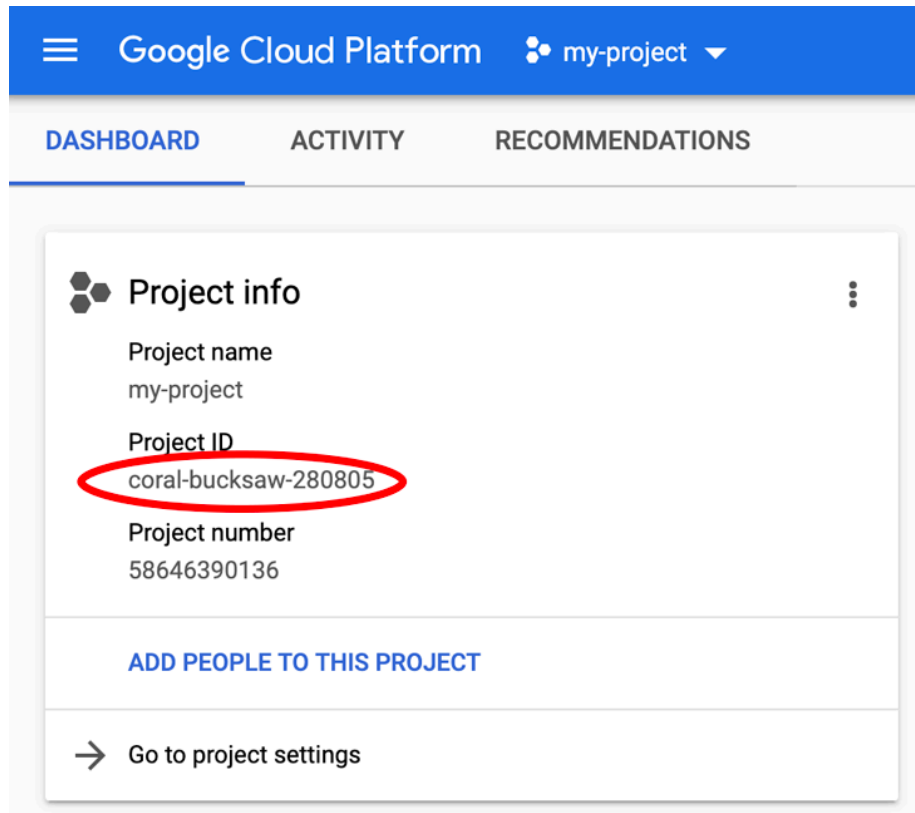
指令輸出

```
[core]
project = <PROJECT_ID>
```

如果專案未設定，請發出下列指令：

```
gcloud config set project <PROJECT_ID>
```

在尋找「`PROJECT_ID`」嗎？請檢查您在設定步驟中使用的 ID，或在 Cloud 控制台資訊主頁中尋找：



Cloud Shell 也會預設設定部分環境變數，這些變數在您執行後續指令時可能很有用。

```
echo $GOOGLE_CLOUD_PROJECT
```

指令輸出

```
<PROJECT_ID>
```

最後，設定預設可用區和專案。

```
gcloud config set compute/zone us-central1-f
```

你可以選擇各種不同區域。詳情請參閱「[地區和區域](#)」。

注意：如果在自己的機器上執行 `gcloud`，各個工作階段都會保留這些設定。但在 Cloud Shell 中，每次啟動新工作階段或重新連線時，都必須重新設定。

前往語言設定

在本程式碼研究室中，所有原始碼都使用 Go。在 Cloud Shell 執行下列指令，確認 Go 版本是否為 1.17 以上

```
go version
```

指令輸出


```
go version go1.18.3 linux/amd64
```

設定 Google Kubernetes 叢集

在本程式碼研究室中，您會在 Google Kubernetes Engine (GKE) 上執行微服務叢集。本程式碼研究室的流程如下：

1. 將基準專案下載至 Cloud Shell
2. 將微服務建構到容器中
3. 將容器上傳至 Google Artifact Registry (GAR)
4. 將容器部署至 GKE
5. 修改服務的原始碼，以便進行追蹤檢測
6. 前往步驟 2

啟用 Kubernetes Engine

首先，我們要設定 Kubernetes 叢集，讓 Shakesapp 在 GKE 上執行，因此需要啟用 GKE。前往「Kubernetes Engine」選單，然後按下「啟用」按鈕。



Kubernetes Engine API

Google

Builds and manages container-based applications, powered by the open source Kubernetes technology.

ENABLE

TRY THIS API [↗](#)

現在可以開始建立 Kubernetes 叢集。

注意：您也可以使用下列 gcloud 指令啟用 Kubernetes Engine API。

```
$ gcloud services enable container.googleapis.com
```

建立 Kubernetes 叢集

在 Cloud Shell 執行下列指令，建立 Kubernetes 叢集。請**確認可用區值**位於您要用於建立 [Artifact Registry](#) 存放區的區域**下方**。如果存放區區域未涵蓋該區域，請變更區域值 `us-central1-f`。

```
gcloud container clusters create otel-trace-codelab2 \  
--zone us-central1-f \  
--release-channel rapid \  
--preemptible \  
--enable-autoscaling \  
--max-nodes 8 \  
--no-enable-ip-alias \  
--scopes cloud-platform
```

指令輸出

```
Note: Your Pod address range (`--cluster-ipv4-cidr`) can accommodate at most 1008  
node(s).  
Creating cluster otel-trace-codelab2 in us-central1-f... Cluster is being health-  
checked (master is healthy)...done.  
Created [https://container.googleapis.com/v1/projects/development-215403/zones/us-  
central1-f/clusters/otel-trace-codelab2].  
To inspect the contents of your cluster, go to:  
https://console.cloud.google.com/kubernetes/workload\_/gcloud/us-central1-f/otel-  
trace-codelab2?project=development-215403  
kubeconfig entry generated for otel-trace-codelab2.  
NAME: otel-trace-codelab2  
LOCATION: us-central1-f  
MASTER_VERSION: 1.23.6-gke.1501  
MASTER_IP: 104.154.76.89  
MACHINE_TYPE: e2-medium  
NODE_VERSION: 1.23.6-gke.1501  
NUM_NODES: 3  
STATUS: RUNNING
```

設定 Artifact Registry 和 Skaffold

現在我們有可供部署的 Kubernetes 叢集。接著，我們要準備容器登錄檔，以便推送及部署容器。在這些步驟中，我們需要設定 Artifact Registry (GAR) 和 Skaffold，以便使用。

設定 Artifact Registry

前往「Artifact Registry」選單，然後按下「啟用」按鈕。



Artifact Registry API

Google

ENABLE

TRY THIS API

注意：您也可以使用下列 gcloud 指令啟用 Kubernetes Engine API。

```
$ gcloud services enable artifactregistry.googleapis.com
```

稍待片刻，您會看到 GAR 的存放區瀏覽器。按一下「CREATE REPOSITORY」按鈕，然後輸入存放區名稱。

Artifact Registry Repositories + CREATE REPOSITORY DELETE SETUP INSTRUCTIONS

AR Docker repositories have pricing associated with them. [Please see the pricing details in our documentation.](#) Maven and npm repositories

DISMISS GO TO DOCUMENTATION

Filter Enter property name or value

Name	Format	Location	Description	Labels	Encryption	Encryption key	Created	Updated
No rows to display								

在本程式碼研究室中，我將新存放區命名為 `trace-codelab`。構件格式為「Docker」，位置類型為「區域」。選擇的區域應與 Google Compute Engine 預設區域相近。舉例來說，上方的範例選擇了「us-central1-f」，因此這裡我們選擇「us-central1 (Iowa)」。然後按一下「建立」按鈕。

Name *
trace-codelab

Format

☒ Docker

☐ Maven ALPHA

☐ npm ALPHA

☐ APT ALPHA

Location type

☒ Region

☐ Multi-region

Region *
us-central1 (Iowa)

Description

Labels

+ ADD LABEL

Encryption

☒ Google-managed encryption key
No configuration required

☐ Customer-managed encryption key (CMEK)
Manage via Google Cloud Key Management Service

CREATE **CANCEL**

現在，您會在存放區瀏覽器中看到「trace-codelab」。

Filter Enter property name or value				
☐	Name ↑	Format	Location	Description
☐	trace-codelab	 Docker	us-central1 (Iowa)	

我們稍後會回到這裡檢查登錄路徑。

Skaffold 設定

如果您要建構在 Kubernetes 上執行的微服務，[Skaffold](#) 是非常實用的工具。只要使用一小組指令，即可處理建構、推送及部署應用程式容器的工作流程。Skaffold 預設會使用 Docker Registry 做為容器登錄檔，因此您需要設定 Skaffold，讓它在將容器推送至 GAR 時能夠辨識。

再次開啟 Cloud Shell，確認是否已安裝 Skaffold。(Cloud Shell 預設會將 skaffold 安裝至環境中)。執行下列指令，查看 Skaffold 版本。

```
skaffold version
```

指令輸出

```
v1.38.0
```

注意：如果您是在 Cloud Shell 以外的環境中執行程式碼研究室，請自行安裝 Skaffold。 `gcloud components install skaffold` 可讓您安裝 Skaffold。

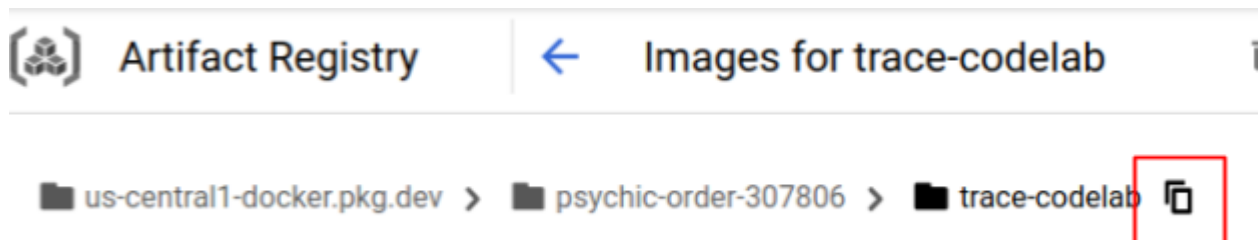
現在，您可以註冊供 Skaffold 使用的預設存放區。如要取得登錄路徑，請前往 Artifact Registry 資訊主頁，然後點選您在上一個步驟中設定的存放區名稱。

Filter Enter property name or value				
☐	Name ↑	Format	Location	Desc
☐	 trace-codelab	 Docker	us-central1 (Iowa)	

頁面頂端就會顯示導覽標記路徑。按一下



圖示，將登錄檔路徑複製到剪貼簿。



按一下複製按鈕後，瀏覽器底部會顯示對話方塊，並顯示類似下列內容的訊息：

已複製「us-central1-docker.pkg.dev/psychic-order-307806/trace-codelab」

返回 Cloud Shell。執行 `scaffold config set default-repo` 指令，並使用您剛才從資訊主頁複製的值。

```
scaffold config set default-repo us-central1-docker.pkg.dev/psychic-order-307806/trace-codelab
```

指令輸出

```
set value default-repo to us-central1-docker.pkg.dev/psychic-order-307806/trace-codelab for context gke_stackdriver-sandbox-3438851889_us-central1-b_stackdriver-sandbox
```

此外，您也需要將登錄檔設定為 Docker 設定。執行下列指令：

```
gcloud auth configure-docker us-central1-docker.pkg.dev --quiet
```

指令輸出

```
{
  "credHelpers": {
    "gcr.io": "gcloud",
    "us.gcr.io": "gcloud",
    "eu.gcr.io": "gcloud",
    "asia.gcr.io": "gcloud",
    "staging-k8s.gcr.io": "gcloud",
    "marketplace.gcr.io": "gcloud",
    "us-central1-docker.pkg.dev": "gcloud"
  }
}
Adding credentials for: us-central1-docker.pkg.dev
```

您現在可以進行下一個步驟，在 GKE 上設定 Kubernetes 容器。

摘要

在這個步驟中，您會設定程式碼研究室環境：

- 設定 Cloud Shell
- 為容器登錄建立 Artifact Registry 存放區
- 設定 Scaffold 以使用 Container Registry
- 建立 Kubernetes 叢集，讓 Codelab 微服務在其中執行

下一步

在下一個步驟中，您將建構、推送微服務，並部署到叢集

步驟 3 · 約 0 分鐘

3. 建構、推送及部署微服務

下載程式碼研究室教材

在上一個步驟中，我們已設定本程式碼研究室的所有必要條件。現在您可以在這些服務上執行整個微服務。程式碼研究室資料位於 GitHub，因此請使用下列 Git 指令將資料下載至 Cloud Shell 環境。

```
cd ~  
git clone https://github.com/ymotongpoo/opentelemetry-trace-codelab-go.git  
cd opentelemetry-trace-codelab-go
```

專案的目錄結構如下：

```
.
├── README.md
├── step0
│   ├── manifests
│   ├── proto
│   ├── skaffold.yaml
│   └── src
├── step1
│   ├── manifests
│   ├── proto
│   ├── skaffold.yaml
│   └── src
├── step2
│   ├── manifests
│   ├── proto
│   ├── skaffold.yaml
│   └── src
├── step3
│   ├── manifests
│   ├── proto
│   ├── skaffold.yaml
│   └── src
├── step4
│   ├── manifests
│   ├── proto
│   ├── skaffold.yaml
│   └── src
├── step5
│   ├── manifests
│   ├── proto
│   ├── skaffold.yaml
│   └── src
└── step6
    ├── manifests
    ├── proto
    ├── skaffold.yaml
    └── src
```

- 資訊清單：Kubernetes 資訊清單檔案
- proto：用戶端與伺服器之間通訊的 proto 定義
- src：各項服務的原始碼目錄
- skaffold.yaml：Skaffold 的設定檔

在本程式碼研究室中，您將更新 `step0` 資料夾下的原始碼。您也可以參考 `step[1-6]` 資料夾中的原始碼，瞭解下列步驟的答案。(第 1 部分涵蓋步驟 0 到步驟 4，第 2 部分涵蓋步驟 5 和步驟 6)

執行 skaffold 指令

最後，您已準備好建構、推送及部署整個內容至剛建立的 Kubernetes 叢集。這聽起來像是包含多個步驟，但實際上，Skaffold 會為您完成所有作業。請使用下列指令試試看：

```
cd step0
skaffold dev
```

執行指令後，您會看到 `docker build` 的記錄輸出內容，並確認這些內容已成功推送至登錄檔。

指令輸出

```
...
---> Running in c39b3ea8692b
---> 90932a583ab6
Successfully built 90932a583ab6
Successfully tagged us-central1-docker.pkg.dev/psychic-order-307806/trace-
codelab/serverservice:step1
The push refers to repository [us-central1-docker.pkg.dev/psychic-order-307806/trace-
codelab/serverservice]
cc8f5a05df4a: Preparing
5bf719419ee2: Preparing
2901929ad341: Preparing
88d9943798ba: Preparing
b0fdf826a39a: Preparing
3c9c1e0b1647: Preparing
f3427ce9393d: Preparing
14alca976738: Preparing
f3427ce9393d: Waiting
14alca976738: Waiting
3c9c1e0b1647: Waiting
b0fdf826a39a: Layer already exists
88d9943798ba: Layer already exists
f3427ce9393d: Layer already exists
3c9c1e0b1647: Layer already exists
14alca976738: Layer already exists
2901929ad341: Pushed
5bf719419ee2: Pushed
cc8f5a05df4a: Pushed
step1: digest:
sha256:8acdbe3a453001f120fb22c11c4f6d64c2451347732f4f271d746c2e4d193bbe size: 2001
```

推送所有服務容器後，Kubernetes 部署作業會自動啟動。

指令輸出

```
sha256:b71fce0a96cea08075dc20758ae561cf78c83ff656b04d211ffa00cedb77edf8 size: 1997
Tags used in deployment:
- serverservice -> us-central1-docker.pkg.dev/psychic-order-307806/trace-
codelab/serverservice:step4@sha256:8acdbe3a453001f120fb22c11c4f6d64c2451347732f4f271d
746c2e4d193bbe
- clientservice -> us-central1-docker.pkg.dev/psychic-order-307806/trace-
codelab/clientservice:step4@sha256:b71fce0a96cea08075dc20758ae561cf78c83ff656b04d211f
fa00cedb77edf8
- loadgen -> us-central1-docker.pkg.dev/psychic-order-307806/trace-
codelab/loadgen:step4@sha256:eea2e5bc8463ecf886f958a86906cab896e9e2e380a0eb143deaeaca
40f7888a
Starting deploy...
- deployment.apps/clientservice created
- service/clientservice created
- deployment.apps/loadgen created
- deployment.apps/serverservice created
- service/serverservice created
```

注意：如果收到「No push access to specified image repository」（無法將映像檔推送至指定存放區）等錯誤訊息，請檢查 `skaffold` 指令是否嘗試將映像檔推送至 Docker Hub (docker.io)，而忽略 `skaffold` 中預設存放區的設定。在這種情況下，請嘗試將 `--default-repo` 選項新增至 `"skaffold dev"` 或 `"skaffold run"`，如下所示。

```
$ skaffold run --tail --default-repo=us-central1-docker.pkg.dev/[project ID]/[repository
name]
```

部署完成後，您會在每個容器中看到實際應用程式記錄，這些記錄會發出至 `stdout`，如下所示：

指令輸出

```
[client] 2022/07/14 06:33:15 {"match_count":3040}
[loadgen] 2022/07/14 06:33:15 query 'love': matched 3040
[client] 2022/07/14 06:33:15 {"match_count":3040}
[loadgen] 2022/07/14 06:33:15 query 'love': matched 3040
[client] 2022/07/14 06:33:16 {"match_count":3040}
[loadgen] 2022/07/14 06:33:16 query 'love': matched 3040
[client] 2022/07/14 06:33:19 {"match_count":463}
[loadgen] 2022/07/14 06:33:19 query 'tear': matched 463
[loadgen] 2022/07/14 06:33:20 query 'world': matched 728
[client] 2022/07/14 06:33:20 {"match_count":728}
[client] 2022/07/14 06:33:22 {"match_count":463}
[loadgen] 2022/07/14 06:33:22 query 'tear': matched 463
```

請注意，此時您會看到伺服器傳送的所有訊息。好了，您現在可以開始使用 OpenTelemetry 檢測應用程式，追蹤服務的分布情形。

開始檢測服務前，請先按下 `Ctrl-C` 關閉叢集。

指令輸出

```
...
[client] 2022/07/14 06:34:57 {"match_count":1}
[loadgen] 2022/07/14 06:34:57 query 'what's past is prologue': matched 1
^CCleaning up...
- W0714 06:34:58.464305    28078 gcp.go:120] WARNING: the gcp auth plugin is
deprecated in v1.22+, unavailable in v1.25+; use gcloud instead.
- To learn more, consult https://cloud.google.com/blog/products/containers-kubernetes/kubectl-auth-changes-in-gke
- deployment.apps "clientservice" deleted
- service "clientservice" deleted
- deployment.apps "loadgen" deleted
- deployment.apps "serverservice" deleted
- service "serverservice" deleted
```

摘要

在這個步驟中，您已在環境中準備好程式碼研究室教材，並確認 Skaffold 正常運作。

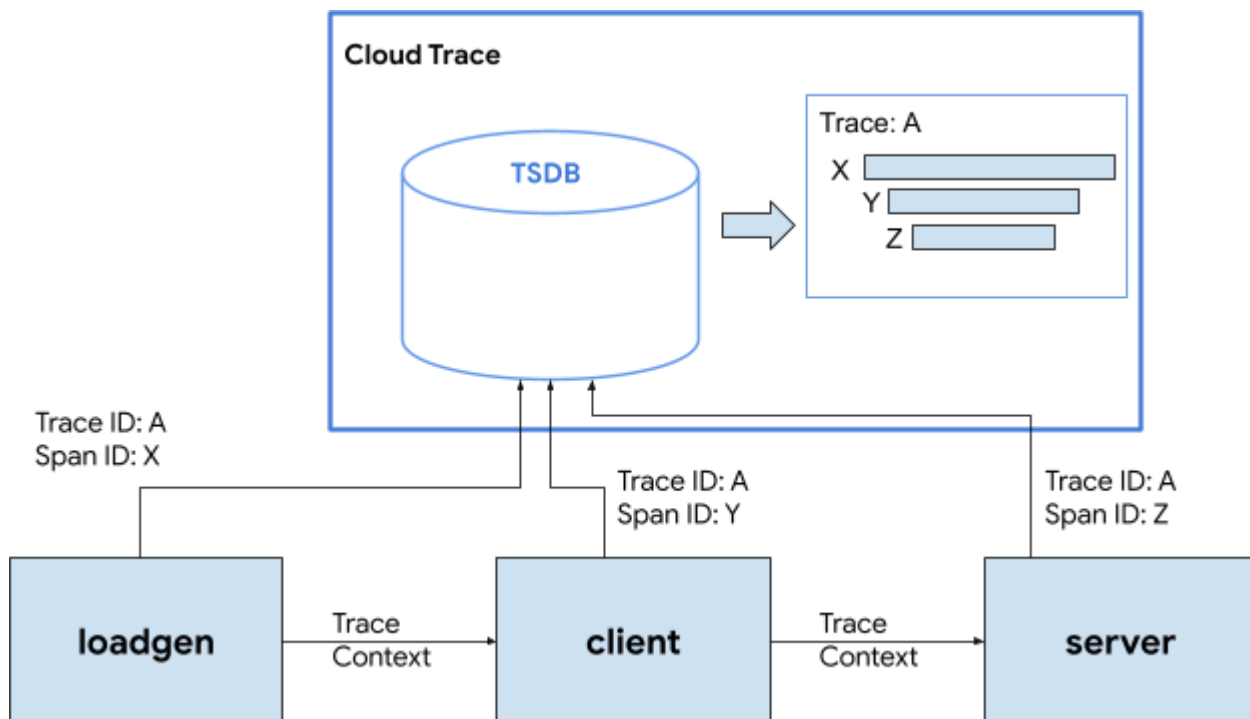
下一步

在下一個步驟中，您將修改 loadgen 服務的原始碼，以檢測追蹤記錄資訊。

4. HTTP 的檢測

追蹤檢測和傳播的概念

編輯原始碼前，請先透過簡單的圖表，瞭解分散式追蹤的運作方式。



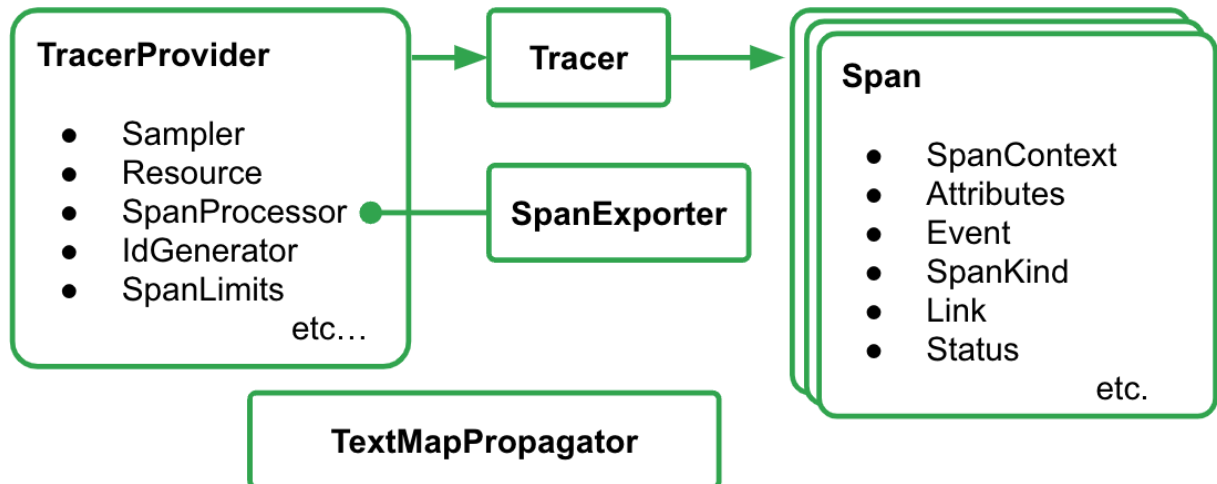
在本範例中，我們會檢測程式碼，將「追蹤記錄」和「時距」資訊匯出至 Cloud Trace，並在從 loadgen 服務到伺服器服務的要求中傳播追蹤記錄結構定義。

應用程式需要傳送追蹤記錄 ID 和時距 ID 等追蹤記錄中繼資料，Cloud Trace 才能將具有相同追蹤記錄 ID 的所有時距組合成一筆追蹤記錄。此外，應用程式也必須在要求下游服務時傳播追蹤內容 (父項時距的追蹤 ID 和時距 ID 組合)，讓下游服務瞭解要處理哪個追蹤內容。

OpenTelemetry 可協助您：

- 產生專屬的追蹤記錄 ID 和範圍 ID
- 將追蹤記錄 ID 和時距 ID 匯出至後端
- 將追蹤內容傳播至其他服務
- 嵌入有助於分析追蹤記錄的額外中繼資料

OpenTelemetry Trace 中的元件



使用 OpenTelemetry 檢測應用程式追蹤記錄的程序如下：

1. 建立匯出工具
2. 建立 TracerProvider，將匯出工具繫結至 1，並設為全域。
3. 設定 TextMapPropagator，以設定傳播方法
4. 從 TracerProvider 取得 Tracer
5. 從追蹤器產生範圍

目前您不需要瞭解每個元件的詳細屬性，但請務必記住以下幾點：

- 這裡的匯出工具可外掛至 TracerProvider
- TracerProvider 會保留所有與追蹤記錄取樣和匯出相關的設定
- 所有追蹤記錄都會封裝在 Tracer 物件中

瞭解這項概念後，我們接著進行實際的編碼工作。

先檢測第一個範圍

檢測負載產生器服務

按一下 Cloud Shell 右上方的



按鈕，開啟 Cloud Shell 編輯器。從左側窗格的檔案總管開啟 `step0/src/loadgen/main.go`，然後找出主要函式。

[step0/src/loadgen/main.go](#)

```

func main() {
    ...
    for range t.C {
        log.Printf("simulating client requests, round %d", i)
        if err := run(numWorkers, numConcurrency); err != nil {
            log.Printf("aborted round with error: %v", err)
        }
        log.Printf("simulated %d requests", numWorkers)
        if numRounds != 0 && i > numRounds {
            break
        }
        i++
    }
}

```

在主函式中，您會看到迴圈呼叫函式 `run`。在目前的實作中，這個區段有 2 行記錄，分別記錄函式呼叫的開始和結束。現在，我們來檢測 Span 資訊，追蹤函式呼叫的延遲時間。

首先，如上一節所述，請設定 OpenTelemetry 的完整設定。新增 OpenTelemetry 套件，如下所示：

step0/src/loadgen/main.go

```

import (
    "context" // step1. add packages
    "encoding/json"
    "fmt"
    "io"
    "log"
    "math/rand"
    "net/http"
    "net/url"
    "time"
    // step1. add packages
    "go.opentelemetry.io/otel"
    "go.opentelemetry.io/otel/attribute"
    stdout "go.opentelemetry.io/otel/exporters/stdout/stdouttrace"
    "go.opentelemetry.io/otel/propagation"
    sdktrace "go.opentelemetry.io/otel/sdk/trace"
    semconv "go.opentelemetry.io/otel/semconv/v1.10.0"
    "go.opentelemetry.io/otel/trace"
    // step1. end add packages
)

```

為了提高可讀性，我們建立名為 `initTracer` 的設定函式，並在 `main` 函式中呼叫該函式。

step0/src/loadgen/main.go

```
// step1. add OpenTelemetry initialization function
func initTracer() (*sdktrace.TracerProvider, error) {
    // create a stdout exporter to show collected spans out to stdout.
    exporter, err := stdout.New(stdout.WithPrettyPrint())
    if err != nil {
        return nil, err
    }

    // for the demonstration, we use AlwaysSample sampler to take all spans.
    // do not use this option in production.
    tp := sdktrace.NewTracerProvider(
        sdktrace.WithSampler(sdktrace.AlwaysSample()),
        sdktrace.WithBatcher(exporter),
    )
    otel.SetTracerProvider(tp)
    otel.SetTextMapPropagator(propagation.TraceContext{})
    return tp, nil
}
```

您可能會發現，設定 OpenTelemetry 的程序與上一節所述相同。在這個實作項目中，我們使用 `stdout` 匯出工具，以結構化格式將所有追蹤資訊匯出至 `stdout`。

然後從主要函式呼叫。呼叫 `initTracer()`，並確保在關閉應用程式時呼叫 `TracerProvider.Shutdown()`。

step0/src/loadgen/main.go

```
func main() {
    // step1. setup OpenTelemetry
    tp, err := initTracer()
    if err != nil {
        log.Fatalf("failed to initialize TracerProvider: %v", err)
    }
    defer func() {
        if err := tp.Shutdown(context.Background()); err != nil {
            log.Fatalf("error shutting down TracerProvider: %v", err)
        }
    }()
    // step1. end setup

    log.Printf("starting worder with %d workers in %d concurrency", numWorkers,
numConcurrency)
    log.Printf("number of rounds: %d (0 is infinite)", numRounds)
    ...
}
```

設定完成後，您需要建立具有專屬追蹤記錄 ID 和時距 ID 的時距。OpenTelemetry 提供實用的程式庫。在儀表 HTTP 用戶端中新增額外的套件。

step0/src/loadgen/main.go

```
import (  
    "context"  
    "encoding/json"  
    "fmt"  
    "io"  
    "log"  
    "math/rand"  
    "net/http"  
    "net/http/httptrace" // step1. add packages  
    "net/url"  
    "time"  
    // step1. add packages  
  
    "go.opentelemetry.io/contrib/instrumentation/net/http/httptrace/otelhttptrace"  
    "go.opentelemetry.io/contrib/instrumentation/net/http/otelhttp"  
    // step1. end add packages  
    "go.opentelemetry.io/otel"  
    "go.opentelemetry.io/otel/attribute"  
    stdout "go.opentelemetry.io/otel/exporters/stdout/stdouttrace"  
    "go.opentelemetry.io/otel/propagation"  
    sdktrace "go.opentelemetry.io/otel/sdk/trace"  
    semconv "go.opentelemetry.io/otel/semconv/v1.10.0"  
    "go.opentelemetry.io/otel/trace"  
)
```

由於負載產生器會在 `runQuery` 函式中以 HTTP 呼叫用戶端服務，因此我們使用 `net/http` 的 `contrib` 套件，並透過 `httptrace` 和 `otelhttp` 套件的擴充功能啟用檢測。 `net/http`

首先，新增套件全域變數 `httpClient`，透過檢測的用戶端呼叫 HTTP 要求。

step0/src/loadgen/main.go

```
var httpClient = http.Client{  
    Transport: otelhttp.NewTransport(http.DefaultTransport)  
}
```

接著，在 `runQuery` 函式中新增檢測，使用 `OpenTelemetry` 和自訂 HTTP 用戶端自動產生的時距，建立自訂時距。請按照下列步驟操作：

1. 從全球 `TracerProvider` 取得 `otel.Tracer()` 的 `Tracer`
2. 使用 `Tracer.Start()` 方法建立根範圍
3. 在任意時間結束根時距 (本例中為 `runQuery` 函式結尾)

step0/src/loadgen/main.go


```

reqURL.RawQuery = v.Encode()
// step1. replace http.Get() with custom client call
// resp, err := http.Get(reqURL.String())

// step1. instrument trace
ctx := context.Background()
tr := otel.Tracer("loadgen")
ctx, span := tr.Start(ctx, "query.request", trace.WithAttributes(
    semconv.TelemetrySDKLanguageGo,
    semconv.ServiceNameKey.String("loadgen.runQuery"),
    attribute.Key("query").String(s),
))
defer span.End()
ctx = httptrace.WithClientTrace(ctx, otelhttptrace.NewClientTrace(ctx))
req, err := http.NewRequestWithContext(ctx, "GET", reqURL.String(), nil)
if err != nil {
    return -1, fmt.Errorf("error creating HTTP request object: %v", err)
}
resp, err := httpClient.Do(req)
// step1. end instrumentation
if err != nil {
    return -1, fmt.Errorf("error sending request to %v: %v",
reqURL.String(), err)
}

```

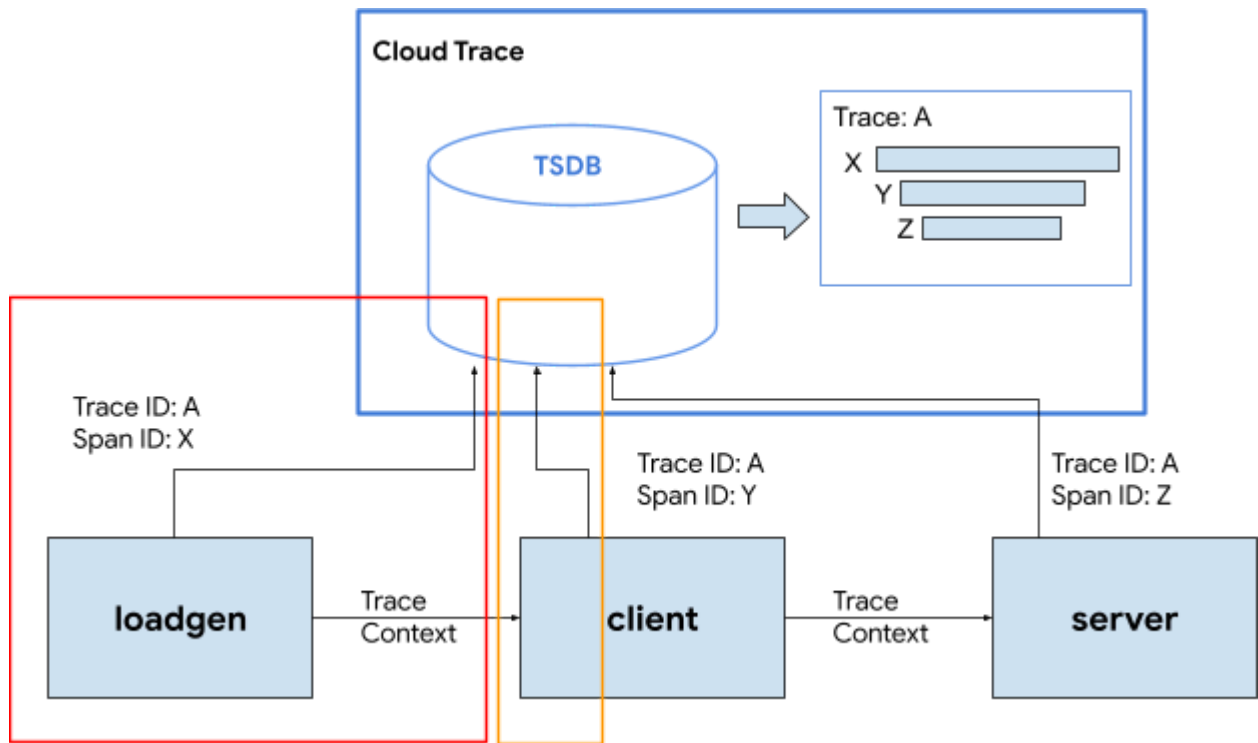
您已完成 loadgen (HTTP 用戶端應用程式) 中的檢測作業。請務必使用 `go mod` 指令更新 `go.mod` 和 `go.sum`。

```
go mod tidy
```

注意：完成這個部分後，每當您在服務中新增套件，就必須執行 `go mod tidy`。

檢測用戶端服務

在上一節中，我們對下圖中以紅色矩形框住的部分進行了插樁。我們在負載產生器服務中，設定了範圍資訊。與負載產生器服務類似，現在我們需要檢測用戶端服務。與負載產生器服務不同的是，用戶端服務需要從 HTTP 標頭中擷取負載產生器服務傳播的追蹤 ID 資訊，並使用該 ID 產生範圍。



開啟 Cloud Shell 編輯器，並新增必要套件，就像我們為負載產生器服務所做的一樣。

step0/src/client/main.go

```
import (
    "context"
    "encoding/json"
    "fmt"
    "io"
    "log"
    "net/http"
    "net/url"
    "os"
    "time"

    "opentelemetry-trace-codelab-go/client/shakesapp"
    // step1. add new import
    "go.opentelemetry.io/contrib/instrumentation/net/http/otelhttp"
    "go.opentelemetry.io/otel"
    "go.opentelemetry.io/otel/attribute"
    stdout "go.opentelemetry.io/otel/exporters/stdout/stdouttrace"
    "go.opentelemetry.io/otel/propagation"
    sdktrace "go.opentelemetry.io/otel/sdk/trace"
    "go.opentelemetry.io/otel/trace"
    "google.golang.org/grpc"
    "google.golang.org/grpc/credentials/insecure"
    // step1. end new import
)
```

同樣地，我們需要設定 OpenTelemetry。只要從 loadgen 複製並貼上 `initTracer` 函式，然後在用戶端服務的 `main` 函式中呼叫該函式即可。

step0/src/client/main.go

```
// step1. add OpenTelemetry initialization function
func initTracer() (*sdktrace.TracerProvider, error) {
    // create a stdout exporter to show collected spans out to stdout.
    exporter, err := stdout.New(stdout.WithPrettyPrint())
    if err != nil {
        return nil, err
    }

    // for the demonstration, we use AlwaysSample sampler to take all spans.
    // do not use this option in production.
    tp := sdktrace.NewTracerProvider(
        sdktrace.WithSampler(sdktrace.AlwaysSample()),
        sdktrace.WithBatcher(exporter),
    )
    otel.SetTracerProvider(tp)
    otel.SetTextMapPropagator(propagation.TraceContext{})
    return tp, nil
}
```

現在可以檢測範圍了。由於用戶端服務需要接受來自 loadgen 服務的 HTTP 要求，因此需要檢測處理常式。用戶端服務中的 HTTP 伺服器是透過 `net/http` 實作，您可以像在 loadgen 中一樣使用 `otelhttp` 套件。

首先，我們將處理常式註冊替換為 `otelhttp` 處理常式。在 `main` 函式中，找出向 `http.HandleFunc()` 註冊 HTTP 處理常式的程式行。

step0/src/client/main.go

```
// step1. change handler to intercept OpenTelemetry related headers
// http.HandleFunc("/", svc.handler)
otelHandler := otelhttp.NewHandler(http.HandlerFunc(svc.handler),
"client.handler")
http.Handle("/", otelHandler)
// step1. end interceptor setting
http.HandleFunc("/_genki", svc.health)
```

接著，我們會在處理常式中實際檢測範圍。找到 `func (*clientService) handler()`，並使用 `trace.SpanFromContext()` 新增範圍檢測。

step0/src/client/main.go

```
func (cs *clientService) handler(w http.ResponseWriter, r *http.Request) {
    ...
    ctx := r.Context()
    ctx, cancel := context.WithCancel(ctx)
    defer cancel()
    // step1. instrument trace
    span := trace.SpanFromContext(ctx)
    defer span.End()
    // step1. end instrument
    ...
}
```

透過這項檢測，您可以取得從 `handler` 方法開頭到結尾的範圍。如要方便分析跨度，請在查詢中新增額外屬性，儲存相符的計數。在記錄行之前加入下列程式碼。

[step0/src/client/main.go](#)

```
func (cs *clientService) handler(w http.ResponseWriter, r *http.Request) {
    ...
    // step1. add span specific attribute
    span.SetAttributes(attribute.Key("matched").Int64(resp.MatchCount))
    // step1. end adding attribute
    log.Println(string(ret))
    ...
}
```

完成上述所有檢測後，您已完成 `loadgen` 和用戶端之間的追蹤檢測。接著來看看實際操作方式。再次使用 `skaffold` 執行程式碼。

```
skaffold dev
```

在 GKE 叢集上執行服務一段時間後，您會看到大量類似下方的記錄訊息：

指令輸出

```

[loadgen] {
[loadgen]     "Name": "query.request",
[loadgen]     "SpanContext": {
[loadgen]         "TraceID": "cfa22247a542beeb55a3434392d46b89",
[loadgen]         "SpanID": "18b06404b10c418b",
[loadgen]         "TraceFlags": "01",
[loadgen]         "TraceState": "",
[loadgen]         "Remote": false
[loadgen]     },
[loadgen]     "Parent": {
[loadgen]         "TraceID": "00000000000000000000000000000000",
[loadgen]         "SpanID": "0000000000000000",
[loadgen]         "TraceFlags": "00",
[loadgen]         "TraceState": "",
[loadgen]         "Remote": false
[loadgen]     },
[loadgen]     "SpanKind": 1,
[loadgen]     "StartTime": "2022-07-14T13:13:36.686751087Z",
[loadgen]     "EndTime": "2022-07-14T13:14:31.849601964Z",
[loadgen]     "Attributes": [
[loadgen]         {
[loadgen]             "Key": "telemetry.sdk.language",
[loadgen]             "Value": {
[loadgen]                 "Type": "STRING",
[loadgen]                 "Value": "go"
[loadgen]             }
[loadgen]         },
[loadgen]         {
[loadgen]             "Key": "service.name",
[loadgen]             "Value": {
[loadgen]                 "Type": "STRING",
[loadgen]                 "Value": "loadgen.runQuery"
[loadgen]             }
[loadgen]         },
[loadgen]         {
[loadgen]             "Key": "query",
[loadgen]             "Value": {
[loadgen]                 "Type": "STRING",
[loadgen]                 "Value": "faith"
[loadgen]             }
[loadgen]         }
[loadgen]     ],
[loadgen]     "Events": null,
[loadgen]     "Links": null,
[loadgen]     "Status": {
[loadgen]         "Code": "Unset",
[loadgen]         "Description": ""
[loadgen]     },

```

```
[loadgen]      "DroppedAttributes": 0,
[loadgen]      "DroppedEvents": 0,
[loadgen]      "DroppedLinks": 0,
[loadgen]      "ChildSpanCount": 5,
[loadgen]      "Resource": [
[loadgen]          {
[loadgen]              "Key": "service.name",
[loadgen]              "Value": {
[loadgen]                  "Type": "STRING",
[loadgen]                  "Value": "unknown_service:loadgen"
[loadgen]              }
[loadgen]          }
[loadgen]      ]
...
```

`stdout` 匯出工具會發出這些訊息。您會發現，`loadgen` 的所有時距父項都有 `□□`，因為這是根時距，也就是追蹤記錄中的第一個時距。 `TraceID: 00000000000000000000000000000000` 您也會發現嵌入屬性 `"query"` 含有傳遞至用戶端服務的查詢字串。

摘要

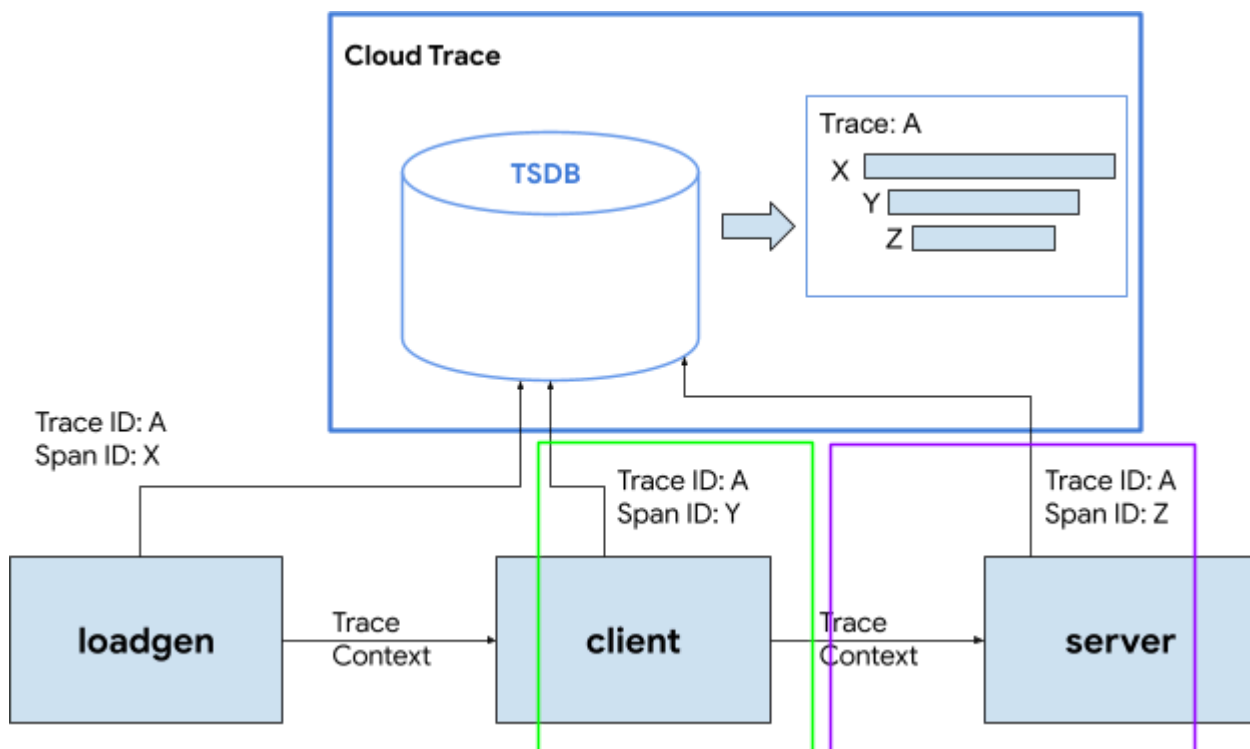
在本步驟中，您已檢測負載產生器服務和以 HTTP 通訊的用戶端服務，並確認您可以在服務之間成功傳播追蹤內容，以及將這兩項服務的 Span 資訊匯出至標準輸出。

下一步

在下一個步驟中，您將檢測用戶端服務和伺服器服務，確認如何透過 gRPC 傳播追蹤內容。

5. gRPC 的檢測

在上一個步驟中，我們在這個微服務中檢測了要求的前半部分。在本步驟中，我們會嘗試在用戶端服務和伺服器服務之間，檢測 gRPC 通訊。(下圖中的綠色和紫色矩形)



預先建構 gRPC 用戶端的檢測作業

OpenTelemetry 生態系統提供許多實用的程式庫，可協助開發人員檢測應用程式。在上一個步驟中，我們使用了 `net/http` 套件的預先建構插樁。在這個步驟中，由於我們嘗試透過 gRPC 傳播追蹤記錄內容，因此使用相關程式庫。

首先，請匯入名為 `otelgrpc` 的預先建構 gRPC 套件。

[step0/src/client/main.go](#)

```

import (
    "context"
    "encoding/json"
    "fmt"
    "io"
    "log"
    "net/http"
    "net/url"
    "os"
    "time"

    "opentelemetry-trace-codelab-go/client/shakesapp"
    // step2. add prebuilt gRPC package (otelgrpc)
    "go.opentelemetry.io/contrib/instrumentation/google.golang.org/grpc/otelgrpc"
    "go.opentelemetry.io/contrib/instrumentation/net/http/otelhttp"
    "go.opentelemetry.io/otel"
    "go.opentelemetry.io/otel/attribute"
    stdout "go.opentelemetry.io/otel/exporters/stdout/stdouttrace"
    "go.opentelemetry.io/otel/propagation"
    sdktrace "go.opentelemetry.io/otel/sdk/trace"
    "go.opentelemetry.io/otel/trace"
    "google.golang.org/grpc"
    "google.golang.org/grpc/credentials/insecure"
)

```

這次用戶端服務是針對伺服器服務的 gRPC 用戶端，因此您需要檢測 gRPC 用戶端。找出 `mustConnGRPC` 函式，並新增 gRPC 攔截器，在用戶端每次向伺服器發出要求時，都會記錄新的範圍。

step0/src/client/main.go


```
// Helper function for gRPC connections: Dial and create client once, reuse.
func mustConnGRPC(ctx context.Context, conn **grpc.ClientConn, addr string) {
    var err error
    // step2. add gRPC interceptor
    interceptorOpt := otelgrpc.WithTracerProvider(otel.GetTracerProvider())
    *conn, err = grpc.DialContext(ctx, addr,
        grpc.WithTransportCredentials(insecure.NewCredentials()),

    grpc.WithUnaryInterceptor(otelgrpc.UnaryClientInterceptor(interceptorOpt)),

    grpc.WithStreamInterceptor(otelgrpc.StreamClientInterceptor(interceptorOpt)),
        grpc.WithTimeout(time.Second*3),
    )
    // step2: end adding interceptor
    if err != nil {
        panic(fmt.Sprintf("Error %s grpc: failed to connect %s", err, addr))
    }
}
```

您已在前一節設定 OpenTelemetry，因此不必執行這項操作。

預先建構的 gRPC 伺服器檢測

如同 gRPC 用戶端，我們呼叫 gRPC 伺服器的預先建構檢測作業。在匯入區段中新增套件，例如：

[step0/src/server/main.go](#)

```
import (  
    "context"  
    "fmt"  
    "io/ioutil"  
    "log"  
    "net"  
    "os"  
    "regexp"  
    "strings"  
  
    "opentelemetry-trace-codelab-go/server/shakesapp"  
  
    "cloud.google.com/go/storage"  
    // step2. add OpenTelemetry packages including otelgrpc  
    "go.opentelemetry.io/contrib/instrumentation/google.golang.org/grpc/otelgrpc"  
    "go.opentelemetry.io/otel"  
    stdout "go.opentelemetry.io/otel/exporters/stdout/stdouttrace"  
    "go.opentelemetry.io/otel/propagation"  
    sdktrace "go.opentelemetry.io/otel/sdk/trace"  
    "google.golang.org/api/iterator"  
    "google.golang.org/api/option"  
    "google.golang.org/grpc"  
    healthpb "google.golang.org/grpc/health/grpc_health_v1"  
)
```

這是首次檢測伺服器，因此您需要先設定 OpenTelemetry，與我們為 loadgen 和用戶端服務執行的操作類似。

[step0/src/server/main.go](#)

```
// step2. add OpenTelemetry initialization function
func initTracer() (*sdktrace.TracerProvider, error) {
    // create a stdout exporter to show collected spans out to stdout.
    exporter, err := stdout.New(stdout.WithPrettyPrint())
    if err != nil {
        return nil, err
    }
    // for the demonstration, we use AlwaysSample sampler to take all spans.
    // do not use this option in production.
    tp := sdktrace.NewTracerProvider(
        sdktrace.WithSampler(sdktrace.AlwaysSample()),
        sdktrace.WithBatcher(exporter),
    )
    otel.SetTracerProvider(tp)
    otel.SetTextMapPropagator(propagation.TraceContext{})
    return tp, nil
}

func main() {
    ...

    // step2. setup OpenTelemetry
    tp, err := initTracer()
    if err != nil {
        log.Fatalf("failed to initialize TracerProvider: %v", err)
    }
    defer func() {
        if err := tp.Shutdown(context.Background()); err != nil {
            log.Fatalf("error shutting down TracerProvider: %v", err)
        }
    }()
    // step2. end setup
    ...
}
```

接著，您需要新增伺服器攔截器。在 `main` 函式中，找出呼叫 `grpc.NewServer()` 的位置，並將攔截器新增至函式。

[step0/src/server/main.go](#)

```
func main() {  
    ...  
    svc := NewServerService()  
    // step2: add interceptor  
    interceptorOpt := otelgrpc.WithTracerProvider(otel.GetTracerProvider())  
    srv := grpc.NewServer(  
  
    grpc.UnaryInterceptor(otelgrpc.UnaryServerInterceptor(interceptorOpt)),  
  
    grpc.StreamInterceptor(otelgrpc.StreamServerInterceptor(interceptorOpt)),  
    )  
    // step2: end adding interceptor  
    shakesapp.RegisterShakespeareServiceServer(srv, svc)  
    ...  
}
```

執行微服務並確認追蹤記錄

然後使用 `skaffold` 指令執行修改後的程式碼。

```
skaffold dev
```

現在，您會在 `stdout` 上看到一堆範圍資訊。

指令輸出

```

...
[server] {
[server]     "Name": "shakesapp.ShakespeareService/GetMatchCount",
[server]     "SpanContext": {
[server]         "TraceID": "89b472f213a400cf975e0a0041649667",
[server]         "SpanID": "96030dbad0061b3f",
[server]         "TraceFlags": "01",
[server]         "TraceState": "",
[server]         "Remote": false
[server]     },
[server]     "Parent": {
[server]         "TraceID": "89b472f213a400cf975e0a0041649667",
[server]         "SpanID": "cd90cc3859b73890",
[server]         "TraceFlags": "01",
[server]         "TraceState": "",
[server]         "Remote": true
[server]     },
[server]     "SpanKind": 2,
[server]     "StartTime": "2022-07-14T14:05:55.74822525Z",
[server]     "EndTime": "2022-07-14T14:06:03.449258891Z",
[server]     "Attributes": [
...
[server]     ],
[server]     "Events": [
[server]         {
[server]             "Name": "message",
[server]             "Attributes": [
...
[server]             ],
[server]             "DroppedAttributeCount": 0,
[server]             "Time": "2022-07-14T14:05:55.748235489Z"
[server]         },
[server]         {
[server]             "Name": "message",
[server]             "Attributes": [
...
[server]             ],
[server]             "DroppedAttributeCount": 0,
[server]             "Time": "2022-07-14T14:06:03.449255889Z"
[server]         }
[server]     ],
[server]     "Links": null,
[server]     "Status": {
[server]         "Code": "Unset",
[server]         "Description": ""
[server]     },
[server]     "DroppedAttributes": 0,
[server]     "DroppedEvents": 0,

```

```
[server]      "DroppedLinks": 0,
[server]      "ChildSpanCount": 0,
[server]      "Resource": [
[server]          {
...
[server]      ],
[server]      "InstrumentationLibrary": {
[server]          "Name":
"go.opentelemetry.io/contrib/instrumentation/google.golang.org/grpc/otelgrpc",
[server]          "Version": "semver:0.33.0",
[server]          "SchemaURL": ""
[server]      }
[server] }
...
```

您發現自己尚未嵌入任何範圍名稱，並使用 `trace.Start()` 或 `span.SpanFromContext()` 手動建立範圍。但由於 gRPC 攔截器產生了這些跨度，您仍會取得大量跨度。

注意：如果終端機的記錄大小限制導致您無法在 `stdout` 記錄中找到時距，可以將 `stdout` 匯出工具替換為 `noop` 匯出工具，這樣系統就會捨棄所有收集到的時距，且不會執行任何動作。詳情請參閱文件。

<https://pkg.go.dev/go.opentelemetry.io/otel/sdk/trace/tracetest#NewNoopExporter>

摘要

在這個步驟中，您使用 OpenTelemetry 生態系統程式庫的支援，檢測了以 gRPC 為基礎的通訊。

下一步

在下一個步驟中，您將使用 Cloud Trace 視覺化追蹤記錄，並瞭解如何分析收集到的範圍。

6. 使用 Cloud Trace 視覺化追蹤記錄

您已使用 OpenTelemetry 檢測整個系統中的追蹤記錄。到目前為止，您已瞭解如何檢測 HTTP 和 gRPC 服務。雖然您已瞭解如何進行插樁，但還不知道如何分析這些指標。在本節中，您將以 Cloud Trace 匯出工具取代 stdout 匯出工具，並瞭解如何分析追蹤記錄。

使用 Cloud Trace 匯出工具

OpenTelemetry 的強大特性之一就是可外掛。如要將您檢測收集的所有時距視覺化，只要將 stdout 匯出工具換成 Cloud Trace 匯出工具即可。

開啟各項服務的 `main.go` 檔案，然後找出 `initTracer()` 函式。刪除該行，產生 stdout 匯出工具，並改為建立 Cloud Trace 匯出工具。

step0/src/loadgen/main.go

```
import (
    ...
    // step3. add OpenTelemetry for Cloud Trace package
    cloudtrace "github.com/GoogleCloudPlatform/opentelemetry-operations-
go/exporter/trace"
)

// step1. add OpenTelemetry initialization function
func initTracer() (*sdktrace.TracerProvider, error) {
    // step3. replace stdout exporter with Cloud Trace exporter
    // cloudtrace.New() finds the credentials to Cloud Trace automatically
    following the
        // rules defined by
    golang.org/x/oauth2/google.FindDefaultCredentialsWithParams.
    //
    https://pkg.go.dev/golang.org/x/oauth2/google#FindDefaultCredentialsWithParams
    exporter, err := cloudtrace.New()
    // step3. end replacing exporter
    if err != nil {
        return nil, err
    }

    // for the demonstration, we use AlwaysSample sampler to take all spans.
    // do not use this option in production.
    tp := sdktrace.NewTracerProvider(
        sdktrace.WithSampler(sdktrace.AlwaysSample()),
        sdktrace.WithBatcher(exporter),
    )
    otel.SetTracerProvider(tp)
    otel.SetTextMapPropagator(propagation.TraceContext{})
    return tp, nil
}
```

注意：請務必執行 `go mod tidy`，更新 `go.mod` 和 `go.sum`。

您也需要在用戶端和伺服器服務中編輯相同函式。

執行微服務並確認追蹤記錄

編輯完成後，請照常使用 `skaffold` 指令執行叢集。

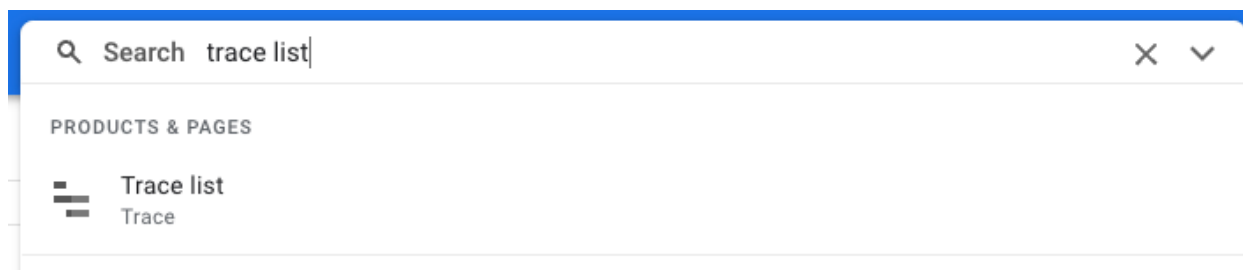
```
skaffold dev
```

現在您應該不會在 `stdout` 上看到太多結構化記錄格式的時距資訊，因為您已將匯出工具換成 Cloud Trace 匯出工具。

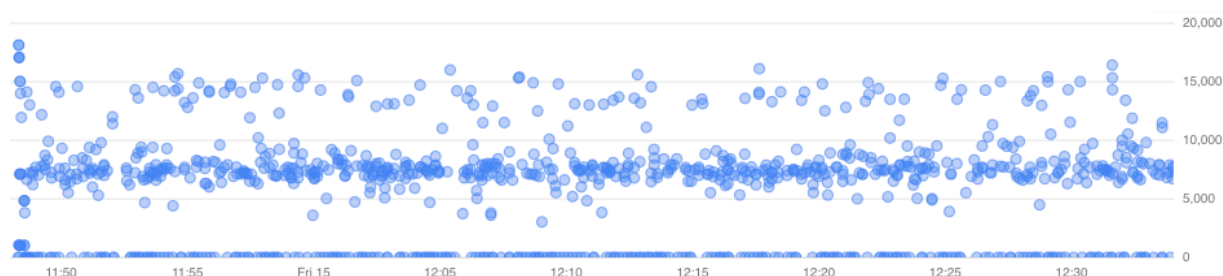
指令輸出

```
[loadgen] 2022/07/14 15:01:07 simulated 20 requests
[loadgen] 2022/07/14 15:01:07 simulating client requests, round 37
[loadgen] 2022/07/14 15:01:14 query 'sweet': matched 958
[client] 2022/07/14 15:01:14 {"match_count":958}
[client] 2022/07/14 15:01:14 {"match_count":3040}
[loadgen] 2022/07/14 15:01:14 query 'love': matched 3040
[client] 2022/07/14 15:01:15 {"match_count":349}
[loadgen] 2022/07/14 15:01:15 query 'hello': matched 349
[client] 2022/07/14 15:01:15 {"match_count":484}
[loadgen] 2022/07/14 15:01:15 query 'faith': matched 484
[loadgen] 2022/07/14 15:01:15 query 'insolence': matched 14
[client] 2022/07/14 15:01:15 {"match_count":14}
[client] 2022/07/14 15:01:21 {"match_count":484}
[loadgen] 2022/07/14 15:01:21 query 'faith': matched 484
[client] 2022/07/14 15:01:21 {"match_count":728}
[loadgen] 2022/07/14 15:01:21 query 'world': matched 728
[client] 2022/07/14 15:01:22 {"match_count":484}
[loadgen] 2022/07/14 15:01:22 query 'faith': matched 484
[loadgen] 2022/07/14 15:01:22 query 'hello': matched 349
[client] 2022/07/14 15:01:22 {"match_count":349}
[client] 2022/07/14 15:01:23 {"match_count":1036}
[loadgen] 2022/07/14 15:01:23 query 'friend': matched 1036
[loadgen] 2022/07/14 15:01:28 query 'tear': matched 463
...
```

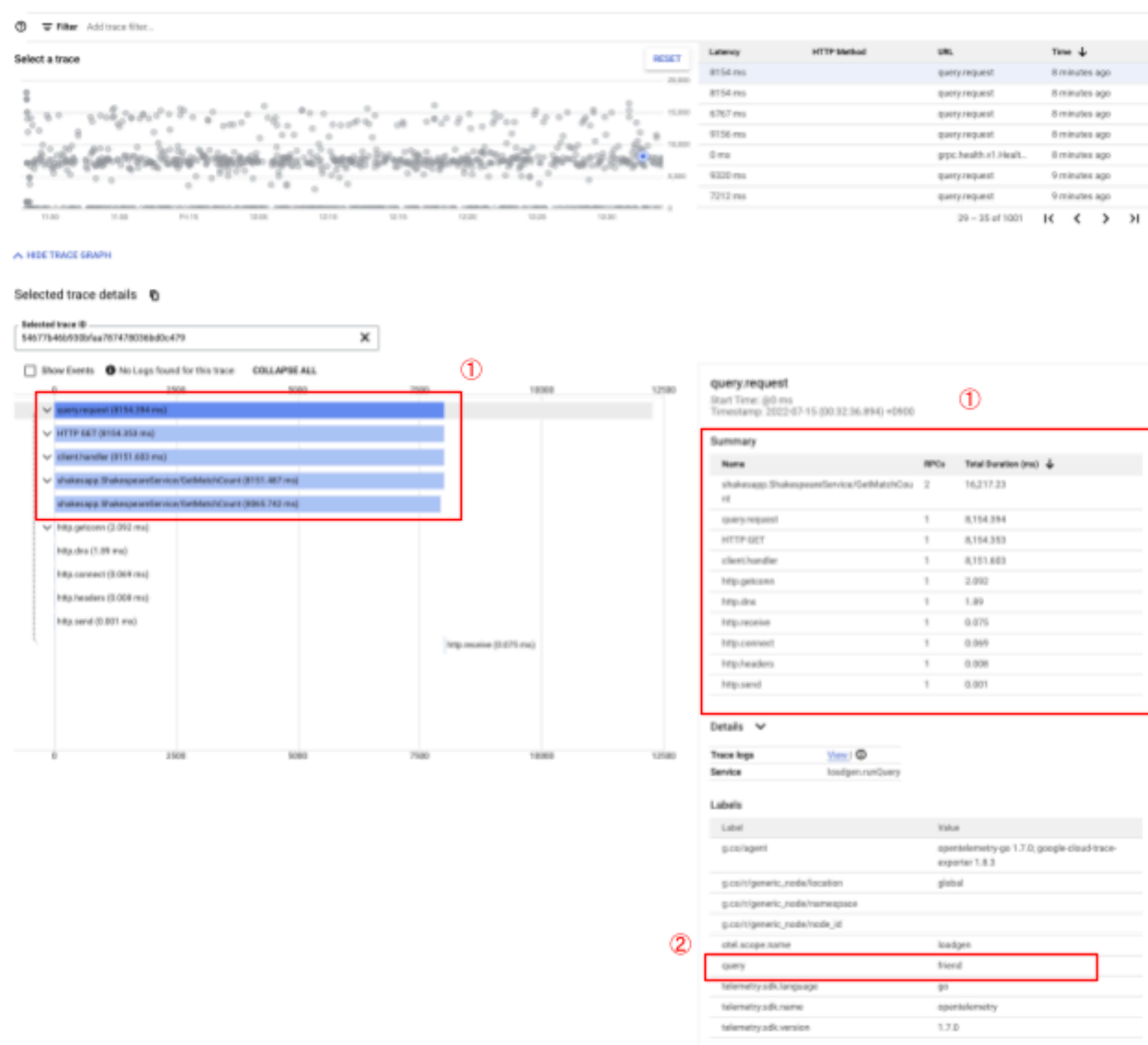
現在請確認所有範圍是否都已正確傳送至 Cloud Trace。存取 Cloud 控制台，然後前往「追蹤記錄清單」。只要在搜尋框中輸入內容，即可輕鬆存取。否則，請按一下左窗格中的選單。



然後您會看到延遲圖表上分布著許多藍點。每個點代表單一追蹤記錄。



按一下其中一個，即可查看追蹤記錄中的詳細資料。



光是快速瀏覽，您就能獲得許多洞察資料。舉例來說，從瀑布圖中，您可以看到延遲的原因主要是因為名為 `shakesapp.ShakespeareService/GetMatchCount` 的範圍。(請參閱上圖中的 1) 您可以從摘要表格確認這點。(最右欄

會顯示每個範圍的持續時間)。此外，這項追蹤記錄是針對「friend」查詢。(請參閱上圖中的 2)

根據這些簡短的分析，您可能會發現需要瞭解 `GetMatchCount` 方法內更精細的範圍。與 `stdout` 資訊相比，視覺化功能更強大。如要進一步瞭解 Cloud Trace 詳細資料，請參閱[官方說明文件](#)。

摘要

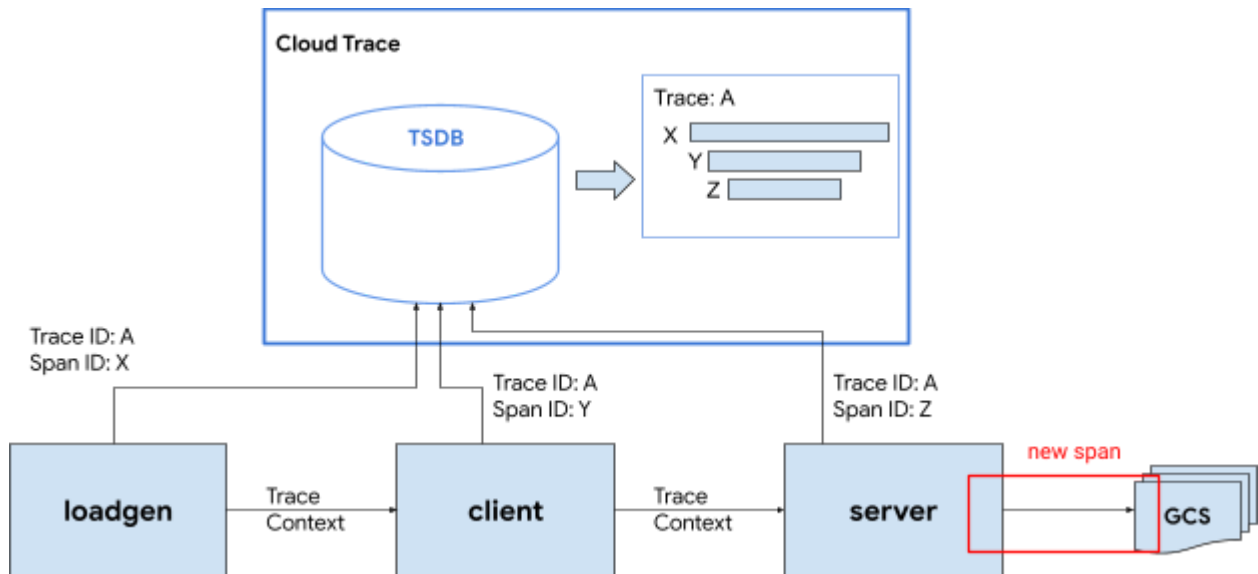
在這個步驟中，您已將 `stdout` 匯出工具換成 Cloud Trace 匯出工具，並在 Cloud Trace 上顯示追蹤記錄。此外，您也學會如何開始分析追蹤記錄。

下一步

在下一個步驟中，您將修改伺服器服務的原始碼，在 `GetMatchCount` 中新增子範圍。

7. 新增子範圍，以便進行更完善的分析

在上一個步驟中，您發現從 loadgen 觀察到的往返時間，主要是伺服器服務中 GetMatchCount 方法 (gRPC 處理常式) 內部的程序所致。不過，由於我們並未對處理常式以外的任何項目進行儀表化，因此無法從瀑布圖中找到進一步的洞察。開始檢測微服務時，這很常見。



在本節中，我們將檢測伺服器呼叫 Google Cloud Storage 的子時距，因為在過程中，某些外部網路 I/O 通常會耗費很長時間，因此找出呼叫是否為原因非常重要。

在伺服器中檢測子時距

在伺服器中開啟 `main.go`，然後找出 `readFiles` 函式。這項函式會呼叫 Google Cloud Storage 的要求，擷取莎士比亞作品的所有文字檔。在這個函式中，您可以建立子範圍，就像您在用戶端服務中為 HTTP 伺服器檢測所做的一樣。

step0/src/server/main.go

```
func readFiles(ctx context.Context, bucketName, prefix string) ([]string, error) {
    type resp struct {
        s    string
        err error
    }

    // step4: add an extra span
    span := trace.SpanFromContext(ctx)
    span.SetName("server.readFiles")
    span.SetAttributes(attribute.Key("bucketname").String(bucketName))
    defer span.End()
    // step4: end add span
    ...
}
```

新增範圍的步驟就到此為止。執行應用程式，看看會發生什麼事。

執行微服務並確認追蹤記錄

編輯完成後，請照常使用 `skaffold` 指令執行叢集。

```
skaffold dev
```

然後從追蹤記錄清單中選取名為「`query.request`」的追蹤記錄。您會看到類似的追蹤瀑布圖，但 `shakesapp.ShakespeareService/GetMatchCount` 下方會出現新的範圍。(下方紅色矩形框住的範圍)



從這張圖表可以看出，對 Google Cloud Storage 的外部呼叫佔用大量延遲時間，但仍有其他項目造成大部分的延遲。

光是查看追蹤瀑布圖，您就已經獲得許多洞察資料。如何取得應用程式的進一步效能詳細資料？此時分析器就會派上用場，但現在，我們就先結束這個 Codelabs，將所有教學課程委派給第 2 部分。

摘要

在這個步驟中，您在伺服器服務中檢測了另一個範圍，並進一步瞭解系統延遲。

步驟 8 · 約 0 分鐘

8. 恭喜

您已成功使用 OpenTelemetry 建立分散式追蹤記錄，並在 Google Cloud Trace 中確認微服務的要求延遲時間。

如要進行延伸練習，可以自行嘗試下列主題。

- 目前的實作方式會傳送健康狀態檢查產生的所有 span。(`grpc.health.v1.Health/Check`) 如何從 Cloud Trace 中篩除這些時距？提示請見[這裡](#)。
- 將事件記錄檔與時距相互關聯，並瞭解這項功能在 Google Cloud Trace 和 Google Cloud Logging 中的運作方式。提示請見[這裡](#)。
- 以其他語言的服務取代部分服務，並嘗試使用該語言的 OpenTelemetry 進行檢測。

此外，如要進一步瞭解剖析器，請繼續閱讀第 2 部分。在這種情況下，您可以略過下方的清理章節。

注意：Google Kubernetes Engine 和 Google Artifact Registry 會持續消耗資源，並產生費用。

清理

完成本程式碼研究室後，請停止 Kubernetes 叢集，並務必刪除專案，以免 Google Kubernetes Engine、Google Cloud Trace 和 Google Artifact Registry 產生非預期的費用。

請先刪除叢集。如果您使用 `scaffold dev` 執行叢集，只要按下 Ctrl-C 即可。如果叢集是透過 `scaffold run` 執行，請執行下列指令：

```
scaffold delete
```

指令輸出

```
Cleaning up...
- deployment.apps "clientservice" deleted
- service "clientservice" deleted
- deployment.apps "loadgen" deleted
- deployment.apps "serverservice" deleted
- service "serverservice" deleted
```

刪除叢集後，請依序選取選單窗格中的「IAM & Admin」>「Settings」，然後按一下「SHUT DOWN」按鈕。

Settings

 MOVE

 SHUT DOWN

Project name

otel-trace-codelab

SAVE

Project ID

psychic-order-307806

Project number:

850914116791

然後在對話方塊的表單中輸入專案 ID (而非專案名稱)，並確認要關閉專案。